Lesson 8

Material

https://github.com/unixfile/learning-python-2026

Today

- Libraries and the Python Package Index
- Importing, and how Python finds libraries
- Installing packages with `uv`
- Virtual environments
- Modules and the module guard pattern
- Packages, `__init__.py` and `__all__`

1 Libraries

- A **library** is reusable code you pull in with `import`
- *Library* is the everyday word; *module* and *package* are the precise terms, coming up later
- Three kinds:
  - Built-in: `math`, `datetime`, `re`, `sys`, `os`, `...`
  - Third-party from PyPI: `requests`, `numpy`, `rich`, `...`
  - Your own modules and packages

The Python Package Index

https://pypi.org

- The public registry of third-party packages

2 Importing a library

```
import re

re.__file__
# '.../lib/python3.14/re/__init__.py'  (path varies)
```

- `import` runs the library once and binds its name
- A library may itself import further libraries
- Imported names sit behind the prefix: `re.search(...)`
- Python looks for the library in the directories on `sys.path`

3 import syntax variants

```
import re                           # re.search(...)
import re as regex                  # regex.search(...)
from re import search               # search(...)
from re import search as find       # find(...)
from re import *                    # everything  (avoid)
```

- A prefix keeps it clear where each name came from
- `from x import *` dumps names into your namespace; avoid it

4 How Python finds libraries

```
import sys
sys.path
# ['', '.../python3.14', ..., '.../site-packages']
```

- `sys.path` is the search path, tried top to bottom
- `''` is the current directory (or the script's directory)
- Then the standard library, then installed third-party packages
- The first match wins

Caveat

- Don't name a file `re.py` or `random.py`: it shadows the real one

5 Adding to the search path

- `sys.path` is just a list, so code can append to it
- Set `$PYTHONPATH` to add directories without editing code
- Those directories appear near the front of `sys.path`

```
# POSIX
export PYTHONPATH=/my/libs:/another/path
# Windows
set PYTHONPATH=C:\my\libs;C:\another\path
```

6 Installing packages with uv

```
uv init myproj          # start a new project
uv add requests         # add a dependency to it
uv remove requests      # remove it again
uv run script.py        # run in the project env
uvx ruff                # run a tool, no install needed
```

- `uv add` records the dependency and installs it
- `uv` resolves versions and downloads them from PyPI
- The legacy `pip install` still works as `uv pip install`

7 Using a package from PyPI

```
uv add rich              # install from PyPI

import rich

rich.print('[bold]hello[/] world')    # styled text
rich.print({'a': 1, 'b': [2, 3]})     # pretty print
```

- `uv add` fetches `rich` from PyPI into your project
- Then `import rich` like any library you've seen
- `rich` adds colour and pretty-printing to the terminal

8 Virtual environments

- An isolated set of packages, kept apart from the system Python
- Each project gets its own, so versions never clash

```
uv add requests    # creates and uses a .venv
uv venv            # or make the .venv explicitly
```

- `uv` manages a `.venv` directory for you, no activation needed
- Older workflow: `python -m venv .venv`, then activate it

9 Modules

- A **module** is a single `.py` file you import elsewhere

```
# greetings.py
def hello(name):
    return f'Hello, {name}!'

import greetings
greetings.hello('Alice')   # 'Hello, Alice!'
```

- The module name is the filename without `.py`
- Importing runs the file once, top to bottom

10 The module guard pattern

```
__name__ == '__main__'     # True when run directly
__name__ == 'greetings'    # True when imported
```

```
# greetings.py
def hello(name):
    return f'Hello, {name}!'

if __name__ == '__main__':
    print(hello('world'))   # only when run directly
```

- A file can be both a runnable script and an importable module
- Code under the guard is skipped when the file is imported

11 The main function pattern

```
def main():
    print(hello('world'))

if __name__ == '__main__':
    main()
```

- Put the script's work in `main()` and call it from the guard
- Keeps temporary variables out of the global scope
- The file stays importable without running `main()`

12 Packages and `__init__.py`

- A **package** is a directory of modules
- Reach a submodule with a dot: `mathpkg.adder`

```
# mathpkg/__init__.py
from .adder import add
from .multiplier import mul
```

```
import mathpkg
mathpkg.add(2, 3)    # 5  (re-exported by __init__.py)
```

- `__init__.py` runs when the package is first imported
- Re-exporting there lets callers skip the submodule path

13 Controlling star imports with `__all__`

```python
# tools.py
__all__ = ['foo', 'bar']   # the public interface

def foo(): ...
def bar(): ...
def _helper(): ...

from tools import *
foo()       # ok
bar()       # ok
_helper()   # NameError: not exported
```

- `__all__` lists what `from module import *` brings in
- Without it, `*` imports every name not starting with `_`

Further reading: subpackages

```
mathpkg/
+-- __init__.py
+-- adder.py
+-- stats/
    +-- __init__.py
    +-- mean.py
```

```
from mathpkg.stats import mean
```

- Packages nest: a subpackage is a directory inside a package
- Each level has its own `__init__.py` by convention

Further reading: running modules with `-m`

```
python -m calendar 2026 6    # run a module as a script
python -m venv .venv         # the stdlib venv way
uv run python -m pytest      # inside the project env
```

- `-m` runs a module from `sys.path` as if it were a script
- The module sees `__name__ == '__main__'`, so its guard fires